

DriveEase

Views Layer

Razor views: the HTML the browser sees

ASP.NET Core 9.0 MVC | Entity Framework Core | PostgreSQL (Supabase)

What the Views layer does

Views are **Razor** (.cshtml) templates that mix HTML with C# to produce the final page. A controller action returns `View(model)`; Razor renders the matching .cshtml file, wraps it in the shared layout, and sends HTML to the browser. Views should contain presentation only - no business logic.

Files in this layer

File	Type	Purpose
_ViewImports.cshtml	Config	Global @using + registers tag helpers for all views.
_ViewStart.cshtml	Config	Sets Layout = "_Layout" for every view.
Shared/_Layout.cshtml	Layout	Master page: , navbar, footer, script includes.
Shared/_Layout.cshtml.css	Style	Scoped CSS for the layout.
Shared/Error.cshtml	View	Error page bound to ErrorViewModel.
Shared/_ValidationScriptsPartial.cshtml	Partial	jQuery validation scripts for forms.
Home/Index.cshtml	View	Landing page.
Home/Privacy.cshtml	View	Privacy page.

Configuration views

_ViewStart.cshtml

Runs before every view and applies the shared layout so individual views do not repeat it.

```
@{
    Layout = "_Layout";
}
```

_ViewImports.cshtml

Namespaces and tag helpers made available to all views without per-file @using.

```
@using wad_project
@using wad_project.Models
@addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers
```

The **tag helpers** line is what enables attributes like asp-controller, asp-action, asp-for and asp-validation-for used in forms and links.

_Layout.cshtml - the master page

Defines the shell every page shares. Key pieces:

Element	What it provides
	Title from ViewData, Bootstrap CSS, site.css, scoped styles.
Navbar	Brand link + Home / Privacy nav (still default 'wad_project' branding).
@RenderBody()	Placeholder where each page's content is injected.
Footer	Copyright + privacy link.
Scripts	jQuery, Bootstrap bundle, site.js, plus optional per-page Scripts section.

```
<title>@ViewData["Title"] - wad_project</title>
...
<main role="main" class="pb-3">
    @RenderBody()
</main>
...
@await RenderSectionAsync("Scripts", required: false)
```

Content views

Home/Index.cshtml

The landing page. Sets its title via ViewData and shows the default template welcome text.

```
@{ ViewData["Title"] = "Home Page"; }  
<div class="text-center">  
    <h1 class="display-4">Welcome</h1>  
    ...  
</div>
```

Home/Privacy.cshtml

Placeholder privacy page; title comes from ViewData and is echoed in the heading.

Shared/Error.cshtml

The only **strongly-typed** view so far. It declares `@model ErrorViewModel` and conditionally shows the request id when present:

```
@model ErrorViewModel  
...  
@if (Model.ShowRequestId)  
{  
    <p><strong>Request ID:</strong> <code>@Model.RequestId</code></p>  
}
```

This shows the standard Razor pattern: **@model** at the top binds the view to a type, and **Model** gives type-safe access to its properties.

Current state & what is missing

The views are still the stock ASP.NET template - only Home and Shared exist, and the layout still shows default 'wad_project' branding. There are **no DriveEase feature views yet** (no vehicle list, booking form, payment page, admin dashboard).

Views a complete build would add

Folder / View	Bound ViewModel	Purpose
Account/Register, Login	RegisterViewModel, LoginViewModel	Auth forms with validation.
Vehicles/Index, Details	VehicleSearchFilterViewModel, Vehicle	Browse/search + detail page.
Vehicles/Recommend	RecommendationRequestViewModel	Recommendation results.
Bookings/Create, MyBookings	BookingRequestViewModel, Booking	Booking flow + history.
Payment/Pay	MakePaymentViewModel	Checkout page.
Reviews/Add	AddReviewViewModel	Leave a review.
Admin/Dashboard	AdminDashboardViewModel	Stats + recent bookings.

Every one of these ViewModels already exists in the ViewModels/ folder with validation attributes, so the views can use asp-for / asp-validation-for immediately once written.